

Avances Pr2_Ad2

Andre Jo 22199, Nelson Escalante 22046

2025-03-07

```
library(GGally)
library(nortest)
library(dplyr)
library(tidyverse)
library(rpart)
library(rpart.plot)
library(caret)
library(discretization)
library(randomForest)
```

```
datos <- read.csv("test.csv")
mergeprice <- read.csv("sample_submission.csv")
train <- read.csv("train.csv")

datos <- merge(datos, mergeprice)

mergeprice <- datos$SalePrice
datos$SalePrice <- NULL
```

1. Conjuntos de Entrenamiento y Prueba

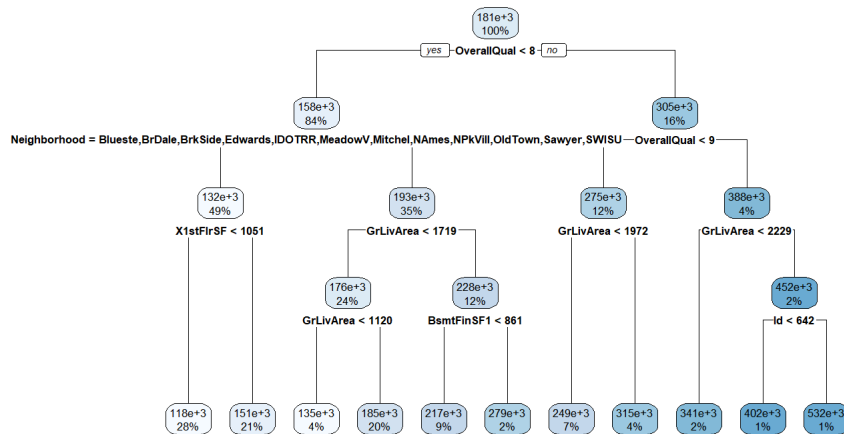
```
#Variables Numericas
#datos[, c("MSSubClass", "LotArea", "OverallQual", "OverallCond",
#         "YearBuilt", "YearRemodAdd", "BsmtFinSF1", "BsmtFinSF2",
#         "BsmtUnfSF", "X1stFlrSF", "X2ndFlrSF", "LowQualFinSF",
#         "BsmtFullBath", "BsmtHalfBath", "FullBath", "HalfBath",
#         "BedroomAbvGr", "KitchenAbvGr", "TotRmsAbvGrd", "Fireplaces",
#         "GarageCars", "GarageArea", "WoodDeckSF", "OpenPorchSF", "EnclosedPorch",
#         "X3SsnPorch", "ScreenPorch", "PoolArea", "MiscVal", "MoSold", "YrSold",
#         "SalePrice")]

datos[] <- lapply(datos, function(x) {
  if (is.numeric(x)) {
    # For numeric columns, replace NAs with the median
    x[is.na(x)] <- median(x, na.rm = TRUE)
  } else if (is.factor(x) | is.character(x)) {
    # For factor or character columns, replace NAs with the mode (most frequent value)
    mode_value <- names(sort(table(x), decreasing = TRUE))[1]
    x[is.na(x)] <- mode_value
  }
  return(x)
})

train[] <- lapply(train, function(x) {
  if (is.numeric(x)) {
    # For numeric columns, replace NAs with the median
    x[is.na(x)] <- median(x, na.rm = TRUE)
  } else if (is.factor(x) | is.character(x)) {
    # For factor or character columns, replace NAs with the mode (most frequent value)
    mode_value <- names(sort(table(x), decreasing = TRUE))[1]
    x[is.na(x)] <- mode_value
  }
  return(x)
})
```

Leemos el conjunto de datos de las casas a observar. También se puede observar que las variables categóricas tienen valores nulos (NA) por lo cual las reemplazamos por valores que son más frecuentes usando la moda. Por otro lado los numéricos que tienen nulos se les aplica la media. Todo esto se aplicó por el conjunto de datos de entrenamiento y de prueba.

2. Elabore un árbol de regresión para predecir el precio de las casas usando todas las variables.



3. Úselo para predecir y analice el resultado. ¿Qué tal lo hizo?

```
predModelo1 <- predict(modelo1, newdata = datos)
rmseModelo1test<-RMSE(predModelo1,mergeprice)
mseModelo1test<-mean((predModelo1 - mergeprice)^2)
predModelo1Train <- predict(modelo1, newdata = train)
rmseModelo1train<-RMSE(predModelo1Train,train$SalePrice)
mseModelo1train<-mean((predModelo1Train - train$SalePrice)^2)

mseModelo1test
```

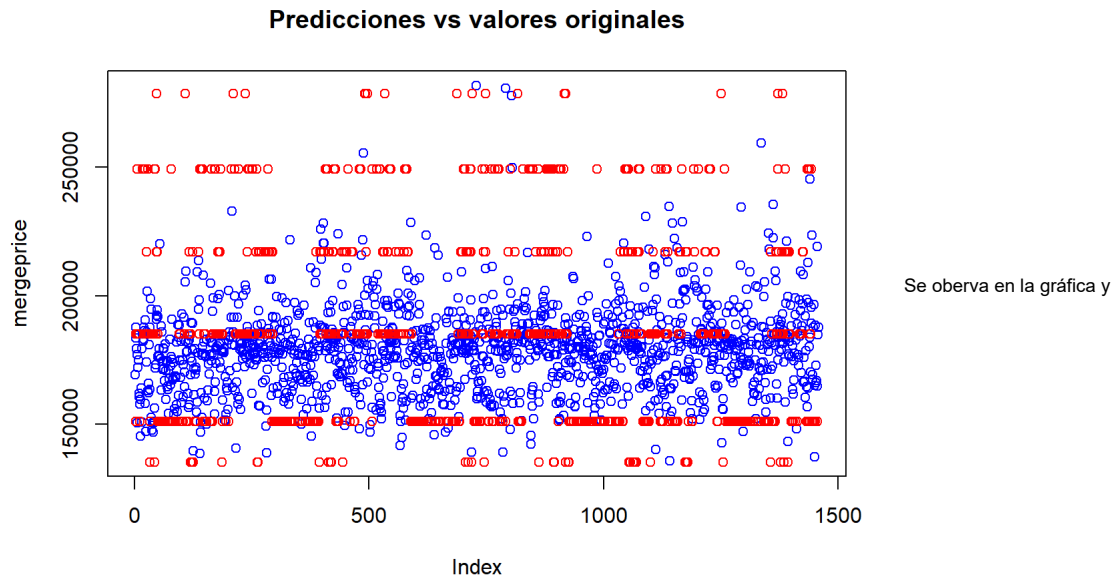
```
## [1] 5953836826
```

```
mseModelo1train
```

```
## [1] 1479488059
```

En cuanto al error medio cuadrado es 5953836826 en el modelo de prueba y 1479488059 en el modelo de entrenamiento. Además, se observa que el rmse del modelo de prueba es de 38464.11 y el rmse del modelo de entrenamiento es de 77161.11 lo cual sus valores son muy altos lo cual puede indicar un overfitting de los datos.

```
plot(mergeprice,col="blue", main="Predicciones vs valores originales")
points(predModelo1, col="red")
legend(30,45,legend=c("original", "prediccion"),col=c("blue", "red"),pch=1, cex=0.8)
```



efectivamente se puede respaldar el overfitting en los datos, como se observa con los puntos rojos, se observa líneas de puntos mientras que en los puntos azules se van espaciando no como la línea de puntos rojos.

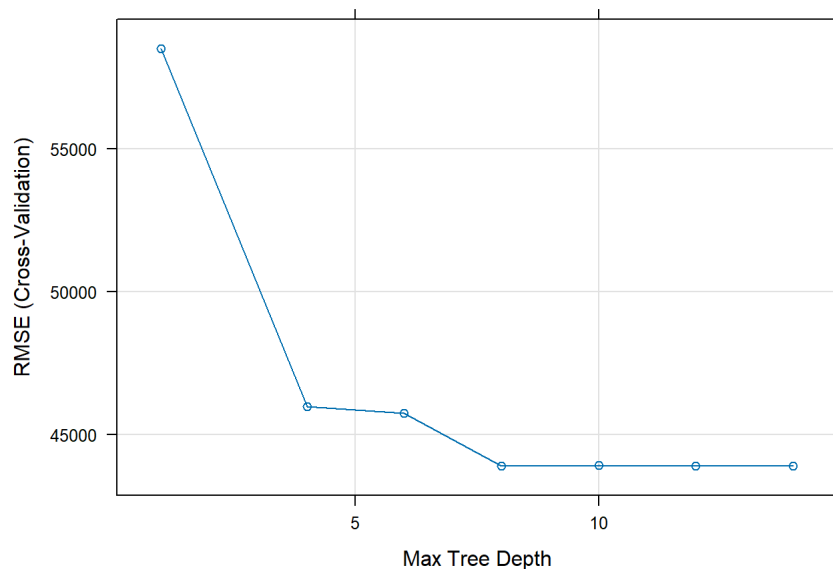
```
controlcv <- trainControl(method = "cv", number = 10)
profundidades <- data.frame(c(1,4,6,8,10,12,14))
names(profundidades)<- "maxdepth"
system.time(
modelo3<-train(SalePrice~.,
  data=train,
  method="rpart2",
  tuneGrid = profundidades,
  trControl = controlcv,
  metric = "RMSE",
  na.action = na.pass
)
)
```

```
## user system elapsed
## 3.02 0.10 3.14
```

```
modelo3
```

```
## CART
##
## 1460 samples
## 80 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1314, 1315, 1312, 1315, 1313, 1314, ...
## Resampling results across tuning parameters:
##
## maxdepth RMSE Rsquared MAE
## 1 58500.34 0.4608829 43338.79
## 4 45974.19 0.6661461 32349.58
## 6 45747.32 0.6684158 31844.94
## 8 43905.53 0.6966329 30160.87
## 10 43923.27 0.6954464 29823.57
## 12 43900.90 0.6962509 29804.11
## 14 43900.90 0.6962509 29804.11
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was maxdepth = 12.
```

```
plot(modelo3)
```

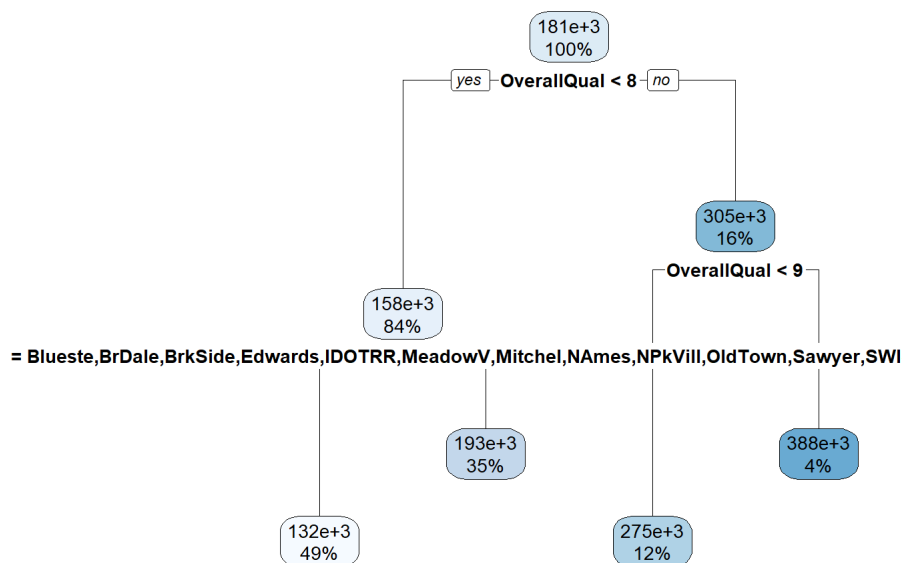


Si tuneamos la profundidad podemos

ver una profundidad máxima de 10. En este caso solo probaremos profundidades menores al árbol anterior.

4. Haga, al menos, 3 modelos más, cambiando el parámetro de la profundidad del árbol. ¿Cuál es el mejor modelo para predecir el precio de las casas?

Modelo con una profundidad a 2



```

predModelo2 <- predict(modelo2, newdata = datos)
rmseModelo2test<-RMSE(predModelo2,mergeprice)
mseModelo2test<-mean((predModelo2 - mergeprice)^2)
predModelo2Train <- predict(modelo2, newdata = train)
rmseModelo2train<-RMSE(predModelo2Train,train$SalePrice)
mseModelo2train<-mean((predModelo2Train - train$SalePrice)^2)

```

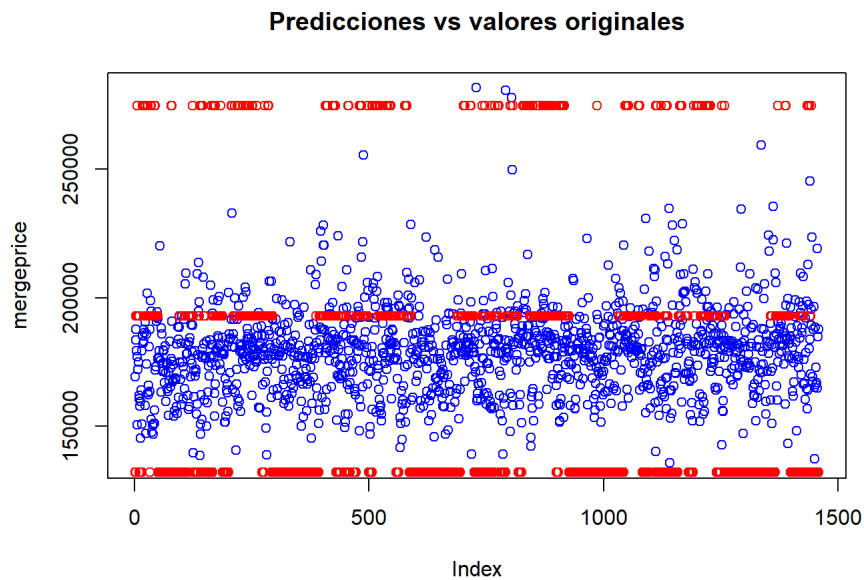
```
mseModelo2test
```

```
## [1] 4638949211
```

```
mseModelo2train
```

```
## [1] 2284545955
```

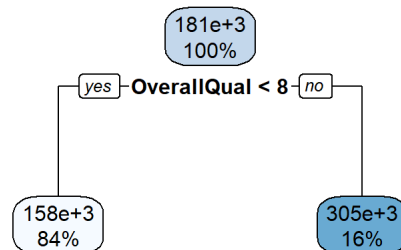
```
plot(mergeprice,col="blue", main="Predicciones vs valores originales")
points(predModelo2, col="red")
legend(30,45,legend=c("original", "prediccion"),col=c("blue", "red"),pch=1, cex=0.8)
```



Se observa que hay overfitting en el

conjunto de datos pero esta vez, la segunda linea si puede predecir ciertos conjuntos de datos mientras que la primera y segunda linea son muy pocos que puede predecir.

Modelo con una profundidad 1



```
predModelo3 <- predict(modelo3, newdata = datos)
rmseModelo3test<-RMSE(predModelo3,mergeprice)
mseModelo3test<-mean((predModelo3 - mergeprice)^2)
predModelo3Train <- predict(modelo3, newdata = train)
rmseModelo3train<-RMSE(predModelo3Train,train$SalePrice)
mseModelo3train<-mean((predModelo3Train - train$SalePrice)^2)

mseModelo3test
```

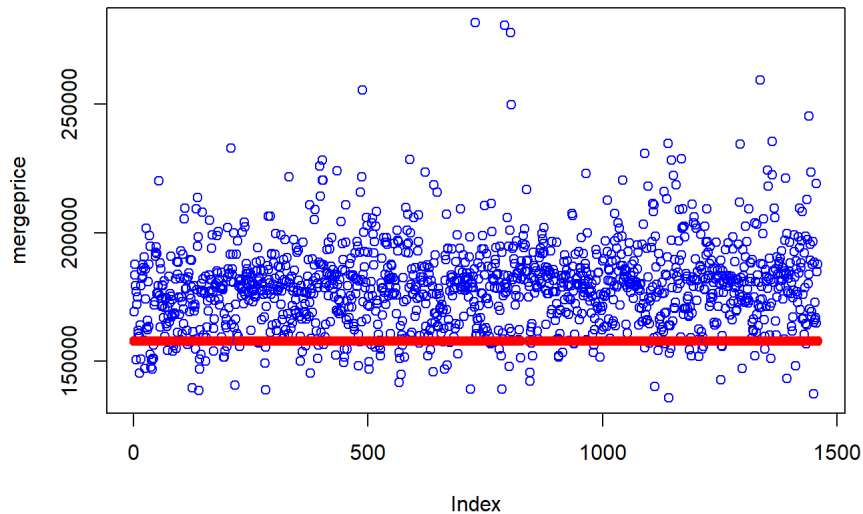
```
## [1] 3278688769
```

```
mseModelo3train
```

[1] 3441133619

```
plot(mergeprice,col="blue", main="Predicciones vs valores originales")
points(predModelo3, col="red")
legend(30,45,legend=c("original", "prediccion"),col=c("blue", "red"),pch=1, cex=0.8)
```

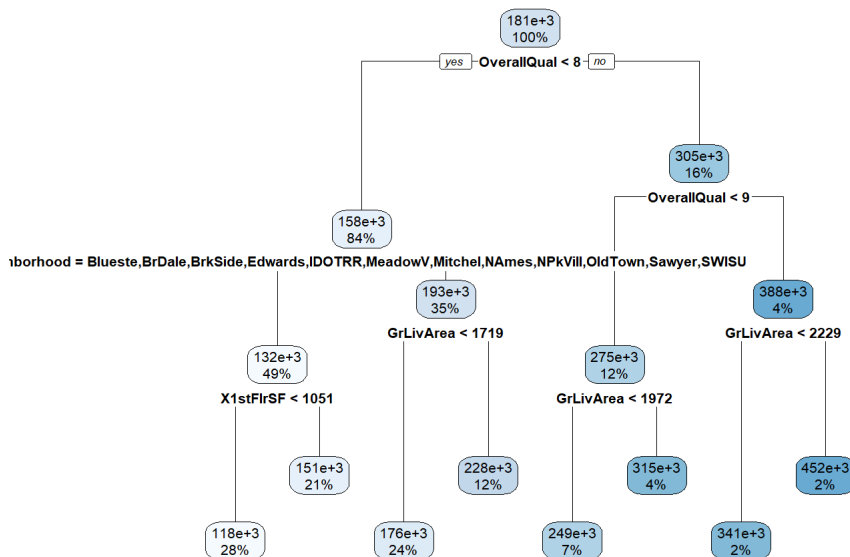
Predicciones vs valores originales



Al igual que la gráfica anterior hay

overfitting de datos.

Modelo con una profundidad 3



```
predModelo4 <- predict(modelo4, newdata = datos)
rmseModelo4test<-RMSE(predModelo4,mergeprice)
mseModelo4test<-mean((predModelo4 - mergeprice)^2)
predModelo4Train <- predict(modelo4, newdata = train)
rmseModelo4train<-RMSE(predModelo4Train,train$SalePrice)
mseModelo4train<-mean((predModelo4Train - train$SalePrice)^2)

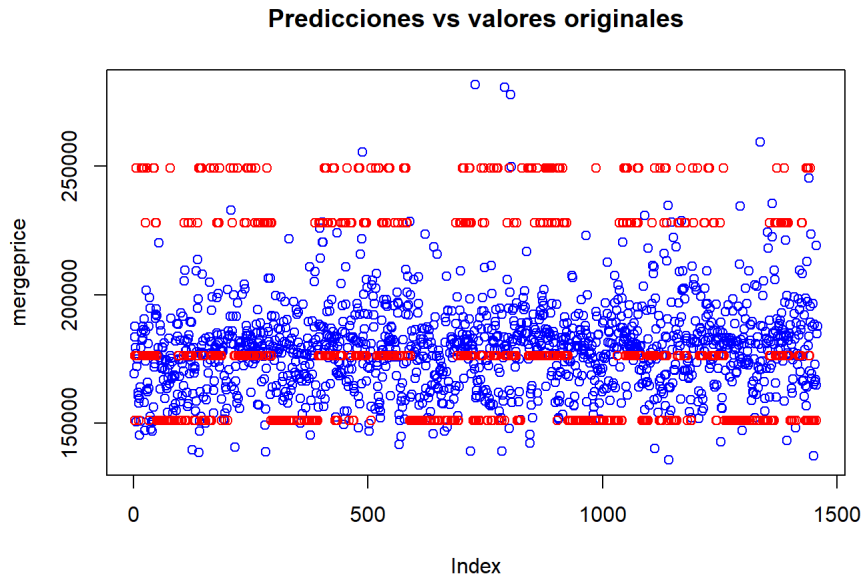
mseModelo4test
```

[1] 4838046177

mseModelo4train

```
## [1] 1702662865
```

```
plot(mergeprice,col="blue", main="Predicciones vs valores originales")
points(predModelo4, col="red")
legend(30,45,legend=c("original", "prediccion"),col=c("blue", "red"),pch=1, cex=0.8)
```



5. Compare los resultados con el modelo de regresión lineal de la hoja anterior, ¿cuál lo hizo mejor?

```
CompModelos<-c(rmseModelo2test,rmseModelo3test,rmseModelo4test, rmseModelo1test)
names(CompModelos)<-c("Modelo1","Modelo2","Modelo3", "Modelo Anterior (1)")
CompModelos
```

##	Modelo1	Modelo2	Modelo3	Modelo Anterior (1)
##	68109.83	57259.84	69556.06	77161.11

La gráfica que tiene una profundidad de 3 puede predecir mejor el precio de las casas debido a que en la última profundidad de la primera gráfica es solo utilizada para clasificar identificadores lo cual no nos provee una información valiosa lo cual podemos quitar. Aunque vemos que todos los modelos tienen overfitting si se puede predecir que modelo puede predecir el precio de las cosas comparando con el rmse. A comparación con la profundidad 4 observamos que hay los dos tienen overfitting de datos pero se observa que el modelo anterior tiene un rmse mejor a comparación con el modelo 3 para poder predecir el precio de las casas.

A continuación se presenta los problemas, 6,7,8 y 9 en el siguiente link

https://github.com/EscasanN/Data_mining_P2/blob/master/Avances2.2/arbo
https://github.com/EscasanN/Data_mining_P2/blob/master/Avances2.2/arbo

10. Entrene un modelo usando validación cruzada, prediga con él. ¿le fue mejor que al modelo anterior?

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

Arbol de Clasifiacion

Nelson Escalante - 22046

Este archivo incluye los incisos del 6 al 8 de la seccion de actividades.

SETUP

```
In [6]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

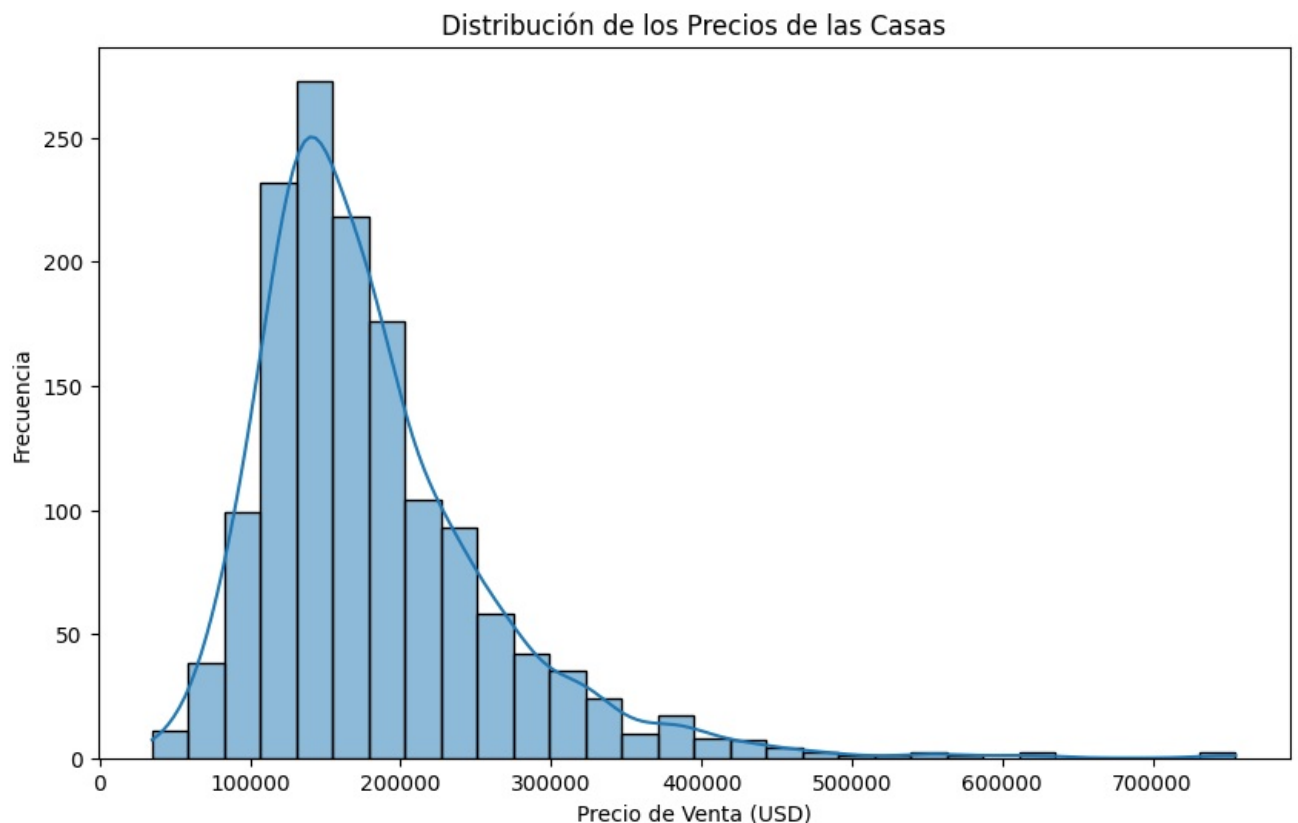
train_path = "../data/train.csv"
train = pd.read_csv(train_path)
```

INCISO 6:

Creacion de la variable respuesta para clasificacion de casas.

Lo principal es observar nuevamente la distribucion de los precios de las casas para entender los datos de buena manera y crear una division apropiada.

```
In [8]: plt.figure(figsize=(10, 6))
sns.histplot(train["SalePrice"], bins=30, kde=True)
plt.title("Distribución de los Precios de las Casas")
plt.xlabel("Precio de Venta (USD)")
plt.ylabel("Frecuencia")
plt.show()
```



Podemos observar que la distribucion de las casas es una distribucion normal con sesgo positivo, lo que nos indica que hay pocas casas que tienen precios muy altos en relacion a las demas.

Para la creacion de nuestra nueva variable tenemos que tener en cuenta esta cualidad de los datos.

Utilizaremos percentiles para crear una division equitativa de los datos en base a los precios y su relacion entre si.

```
In [9]: p33 = train["SalePrice"].quantile(0.33)
p66 = train["SalePrice"].quantile(0.66)

print(f"Límite Económicas: Menos de ${p33:,.0f}")
print(f"Límite Intermedias: Entre ${p33:,.0f} y ${p66:,.0f}")
print(f"Límite Caras: Más de ${p66:,.0f}")
```


Límite Económicas: Menos de \$139,000
Límite Intermedias: Entre \$139,000 y \$189,893
Límite Caras: Más de \$189,893

Con esta nueva division, nuestros datos se distribuyen entre los tres limites de manera equitativa. Vamos a crear la nueva variable con la informacion anterior.

```
In [10]: def categorize_price(price):  
    if price <= p33:  
        return "Económica"  
    elif price <= p66:  
        return "Intermedia"  
    else:  
        return "Cara"  
  
    # Aplicar la función a la columna SalePrice  
    train["PriceCategory"] = train["SalePrice"].apply(categorize_price)  
  
    # Verificar distribución de clases  
    train["PriceCategory"].value_counts()
```

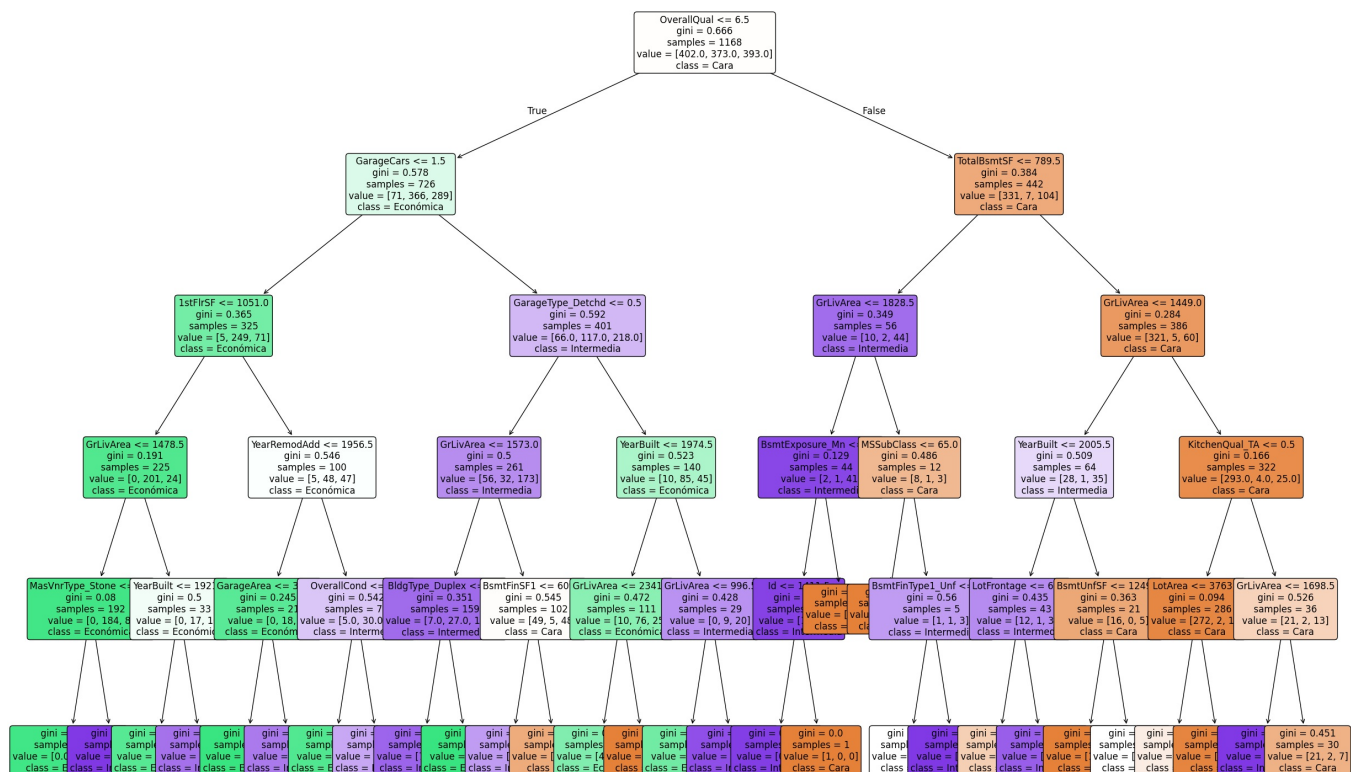
```
Out[10]: PriceCategory  
Cara      497  
Económica  483  
Intermedia 480  
Name: count, dtype: int64
```

Esta nueva variable servira para la creacion del arbol de clasificacion a base de los precios de las casas.

```
In [12]: from sklearn.model_selection import train_test_split  
    from sklearn.tree import DecisionTreeClassifier  
  
    # Seleccionar las variables predictoras (excluyendo SalePrice)  
    X = train.drop(columns=["SalePrice", "PriceCategory"])  
    y = train["PriceCategory"] # Variable objetivo  
  
    # Convertir variables categóricas en numéricas  
    X = pd.get_dummies(X, drop_first=True)  
  
    # Dividir en entrenamiento y prueba (80% - 20%)  
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)  
  
    print(f"Conjunto de entrenamiento: {X_train.shape}")  
    print(f"Conjunto de prueba: {X_test.shape}")  
  
    # Crear el modelo de árbol de clasificación  
    clf = DecisionTreeClassifier(max_depth=5, random_state=42) # Limitamos la profundidad a 5 niveles  
    clf.fit(X_train, y_train) # Entrenar el modelo  
  
    # Predicciones en el conjunto de prueba  
    y_pred = clf.predict(X_test)  
  
Conjunto de entrenamiento: (1168, 245)  
Conjunto de prueba: (292, 245)
```

Es importante visualizar nuestro modelo de manera grafica para evaluarlo.

```
In [18]: import matplotlib.pyplot as plt  
    from sklearn.tree import plot_tree  
  
    plt.figure(figsize=(30, 20))  
    plot_tree(clf, feature_names=X.columns, class_names=clf.classes_, filled=True, rounded=True, fontsize=12)  
    plt.title("Árbol de Clasificación para Categoría de Precio")  
    plt.show()
```



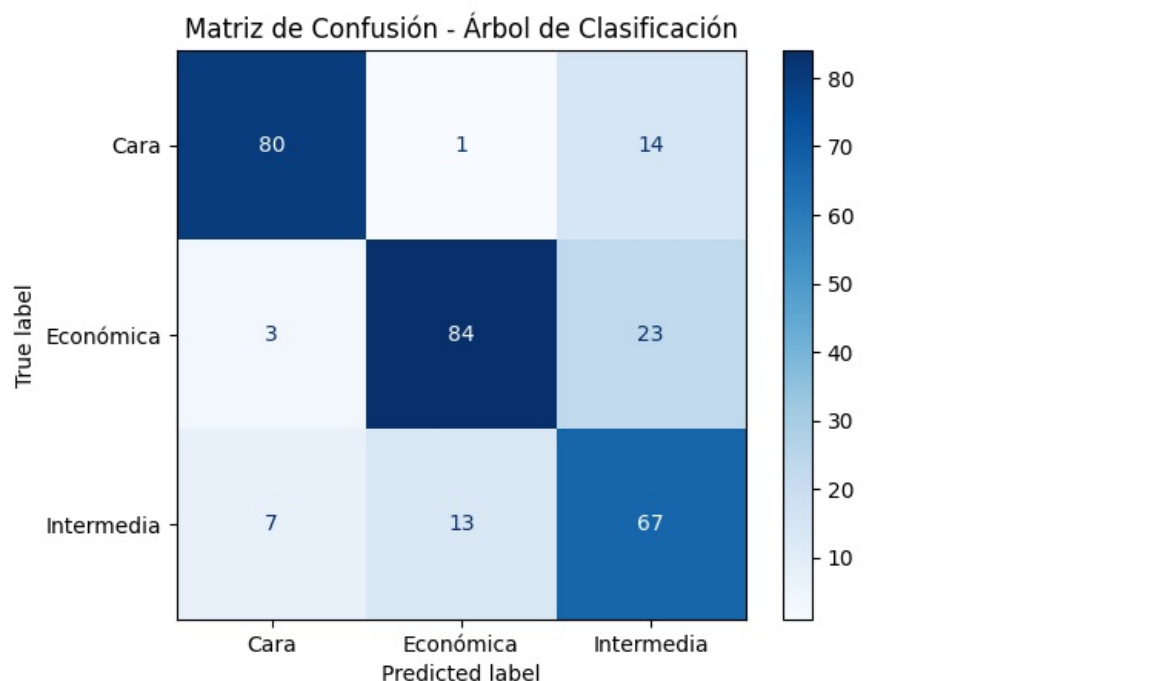
Ahora que tenemos nuestro modelo de clasificación, podemos evaluarlo para verificar que tan bien funciona.

```
In [19]: from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Crear la matriz de confusión
cm = confusion_matrix(y_test, y_pred, labels=clf.classes_)

# Graficar la matriz de confusión
plt.figure(figsize=(8, 6))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=clf.classes_)
disp.plot(cmap="Blues", values_format="d")
plt.title("Matriz de Confusión - Árbol de Clasificación")
plt.show()
```

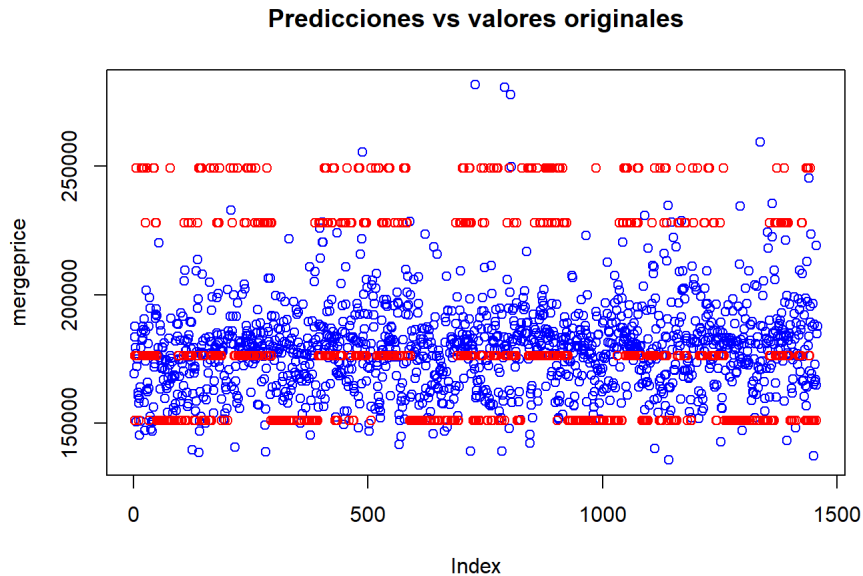
<Figure size 800x600 with 0 Axes>



El modelo ha clasificado bastante bien los datos, por lo que podemos decir que funciona bastante bien para la predicción de los precios.

```
## [1] 1702662865
```

```
plot(mergeprice,col="blue", main="Predicciones vs valores originales")
points(predModelo4, col="red")
legend(30,45,legend=c("original", "prediccion"),col=c("blue", "red"),pch=1, cex=0.8)
```



5. Compare los resultados con el modelo de regresión lineal de la hoja anterior, ¿cuál lo hizo mejor?

```
CompModelos<-c(rmseModelo2test,rmseModelo3test,rmseModelo4test, rmseModelo1test)
names(CompModelos)<-c("Modelo1","Modelo2","Modelo3", "Modelo Anterior (1)")
CompModelos
```

##	Modelo1	Modelo2	Modelo3	Modelo Anterior (1)
##	68109.83	57259.84	69556.06	77161.11

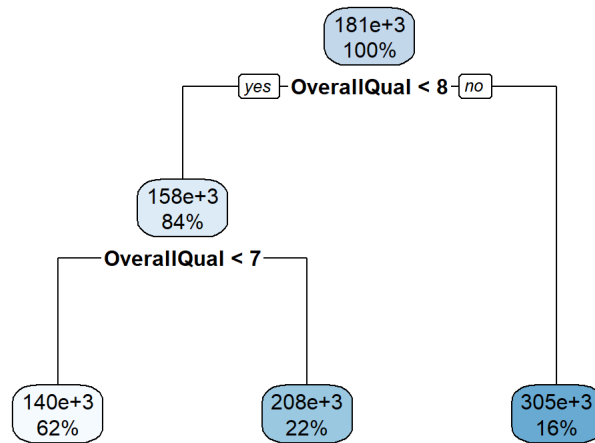
La gráfica que tiene una profundidad de 3 puede predecir mejor el precio de las casas debido a que en la última profundidad de la primera gráfica es solo utilizada para clasificar identificadores lo cual no nos provee una información valiosa lo cual podemos quitar. Aunque vemos que todos los modelos tienen overfitting si se puede predecir que modelo puede predecir el precio de las cosas comparando con el rmse. A comparación con la profundidad 4 observamos que hay los dos tienen overfitting de datos pero se observa que el modelo anterior tiene un rmse mejor a comparación con el modelo 3 para poder predecir el precio de las casas.

A continuación se presenta los problemas, 6,7,8 y 9 en el siguiente link

https://github.com/EscasanN/Data_mining_P2/blob/master/Avances2.2/arbo
https://github.com/EscasanN/Data_mining_P2/blob/master/Avances2.2/arbo

10. Entrene un modelo usando validación cruzada, prediga con él. ¿le fue mejor que al modelo anterior?

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```



```

predModelo5 <- predict(modelo5, newdata = datos)
rmseModelo5test<-RMSE(predModelo5,mergeprice)
mseModelo5test<-mean((predModelo5 - mergeprice)^2)

predModelo5Train <- predict(modelo5, newdata = train)
rmseModelo5train<-RMSE(predModelo5Train,train$SalePrice)
mseModelo5train<-mean((predModelo5Train - train$SalePrice)^2)

```

```
mseModelo5test
```

```
## [1] 3998029436
```

```
mseModelo5train
```

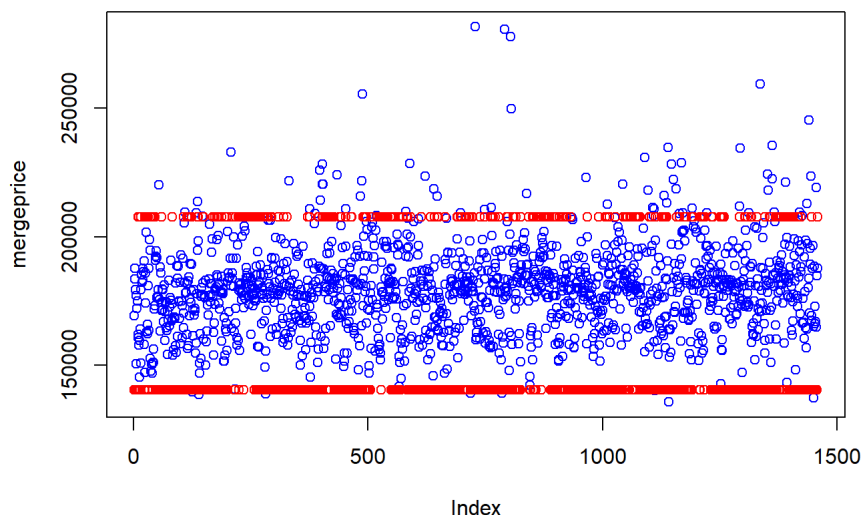
```
## [1] 2707256389
```

```

plot(mergeprice,col="blue", main="Predicciones vs valores originales")
points(predModelo5, col="red")
legend(30,45,legend=c("original", "prediccion"),col=c("blue", "red"),pch=1, cex=0.8)

```

Predicciones vs valores originales



No Dio una mejora con la gráfica

anterior. En base a los datos mostrados, se observa que el modelo no cruzado y cruzado tienen overfitting los datos lo cual solo podemos comparar con el rmse en la cual el modelo no cruzado tiene mayor rmse.

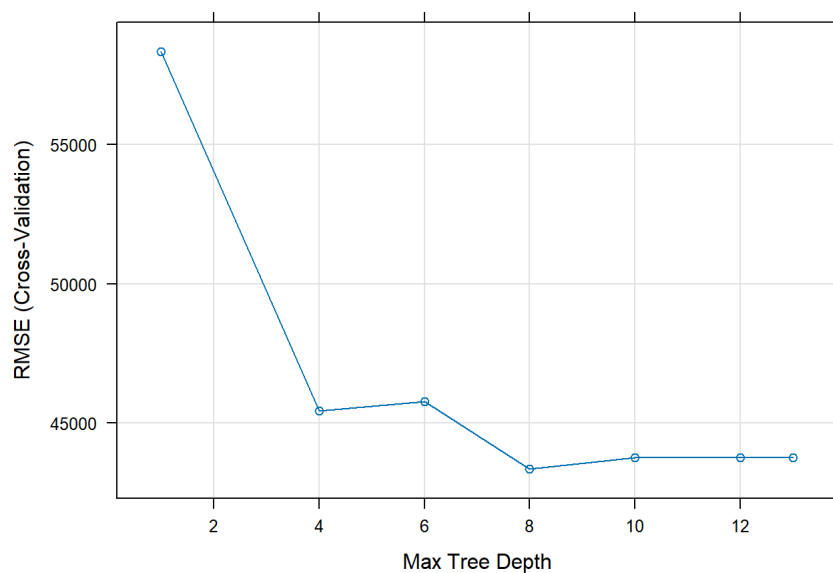
```
CompModelos<-c(rmseModelo1test, rmseModelo4test ,rmseModelo5test)
names(CompModelos)<-c("Modelo no Cruzado", "Modelo no Cruzado con profundidad 3","Modelo Cruzado")
CompModelos
```

```
##           Modelo no Cruzado Modelo no Cruzado con profundidad 3
##           77161.11                69556.06
##           Modelo Cruzado
##           63229.97
```

```
profundidades <- data.frame(c(1,4,6,8,10,12,13))
names(profundidades)<- "maxdepth"
system.time(
modelo6<-train(SalePrice~.,
               data=train,
               method="rpart2",
               #tuneLength = 10,
               tuneGrid = profundidades,
               trControl = controlcv,
               metric = "RMSE",
               na.action = na.pass
               )
)
```

```
## user system elapsed
## 1.97 0.14 2.11
```

```
plot(modelo6)
```



La gráfica de profundidades muestra

que desde la profundidad 4 escala hasta llegar a la profundidad 6 lo cual hay un gran cambio y conforme llegamos a fondo, puede tener una profundidad de 10 donde ya que no se mejora.

```
profundidades <- expand.grid(maxdepth = 2:14)

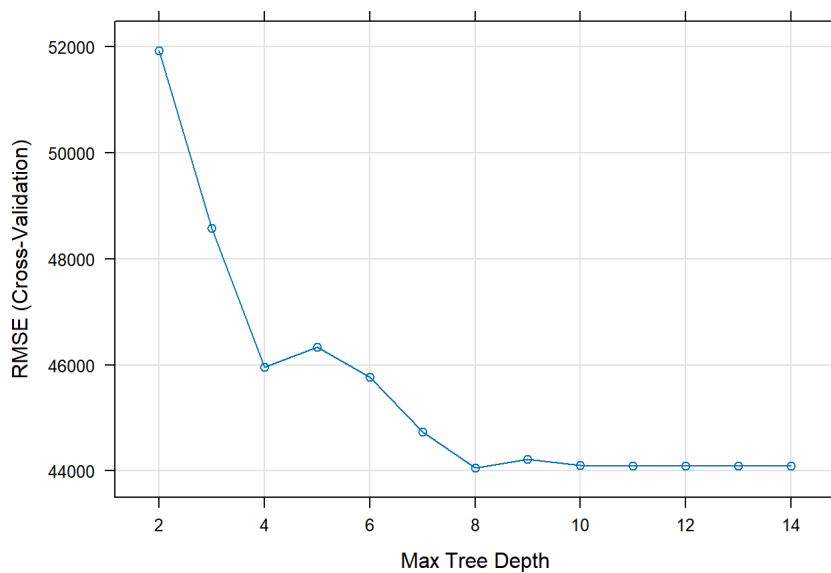
system.time (modelo6.1<-train(SalePrice~.,
                              data=train,
                              method="rpart2",
                              tuneLength = 10,
                              tuneGrid = profundidades,
                              trControl = controlcv,
                              metric = "RMSE",
                              na.action = na.pass
                              )
)
```

```
## user system elapsed
## 2.96 0.09 3.15
```

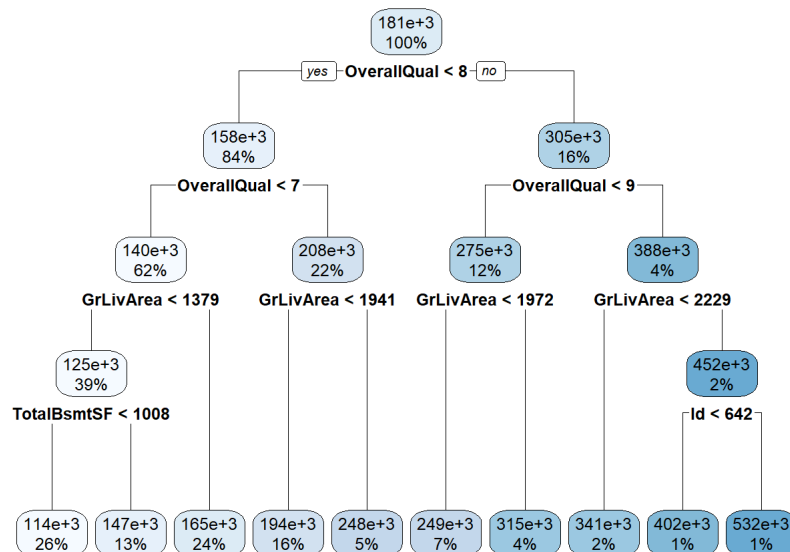
```
modelo6.1
```

```
## CART
##
## 1460 samples
## 80 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1313, 1315, 1316, 1314, 1313, 1314, ...
## Resampling results across tuning parameters:
##
##  maxdepth  RMSE      Rsquared  MAE
##  2         51927.89  0.5745846  36283.65
##  3         48577.77  0.6268160  34692.13
##  4         45955.55  0.6670994  32363.96
##  5         46338.05  0.6598245  32441.21
##  6         45763.18  0.6693950  31726.03
##  7         44736.69  0.6826315  31248.66
##  8         44051.98  0.6926130  30069.31
##  9         44225.54  0.6917890  30070.47
## 10         44109.39  0.6938788  30046.71
## 11         44094.21  0.6942243  29933.43
## 12         44094.21  0.6942243  29933.43
## 13         44094.21  0.6942243  29933.43
## 14         44094.21  0.6942243  29933.43
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was maxdepth = 8.
```

```
plot(modelo6.1)
```



```
rpart.plot(modelo6.1$finalModel)
```



De acuerdo con el modelo de greedy,

la profundidad es la misma lo cual vamos a utilizar una profundidad de 4 para realizar las los modelos.

```

predModelo6.1 <- predict(modelo6.1, newdata = datos)
rmseModelo6.1test<-RMSE(predModelo6.1,mergeprice)
mseModelo6.1test<-mean((predModelo6.1 - mergeprice)^2)

predModelo6.1Train <- predict(modelo6.1, newdata = train)
rmseModelo6.1train<-RMSE(predModelo6.1Train,train$SalePrice)
mseModelo6.1train<-mean((predModelo6.1Train - train$SalePrice)^2)
  
```

```
mseModelo6.1test
```

```
## [1] 5843740252
```

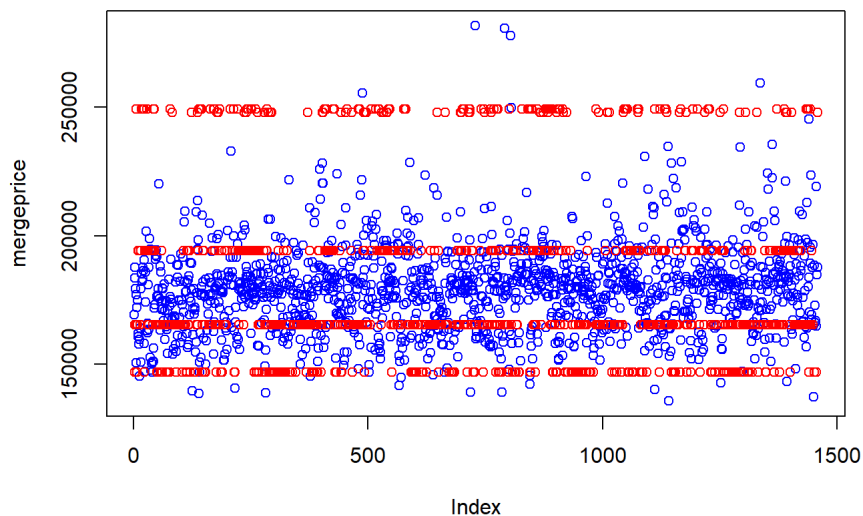
```
mseModelo6.1train
```

```
## [1] 1542464278
```

```

plot(mergeprice,col="blue", main="Predicciones vs valores originales")
points(predModelo6.1, col="red")
legend(30,45,legend=c("original", "prediccion"),col=c("blue", "red"),pch=1, cex=0.8)
  
```

Predicciones vs valores originales



Se observa que todavia existe

overfitting en la gráfica observada.

```

profundidades <- expand.grid(maxdepth = 2:15)

system.time (modelo7.1<-train(SalePrice~.,
                             data=train,
                             method="rpart2",
                             tuneLength = 10,
                             tuneGrid = profundidades,
                             trControl = controlcv,
                             metric = "RMSE",
                             na.action = na.pass
                             )
)

```

```

## user system elapsed
## 2.84 0.12 2.97

```

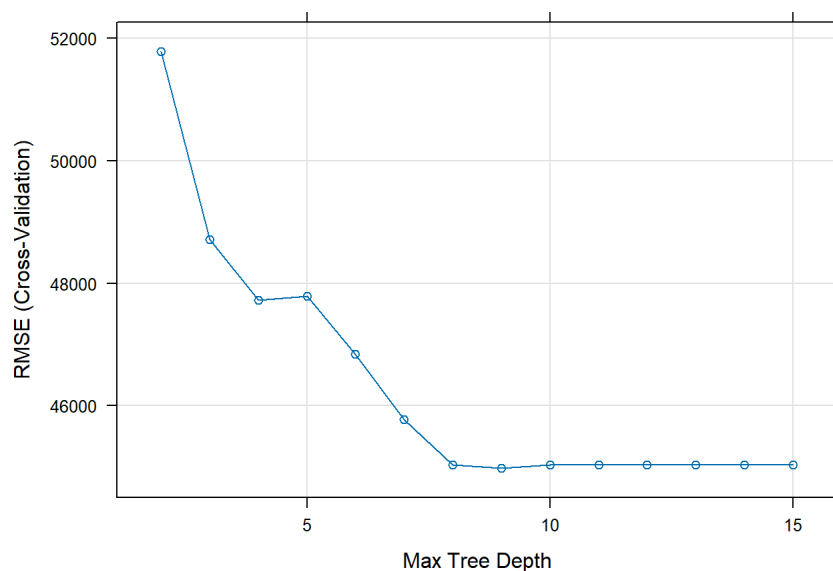
```
modelo7.1
```

```

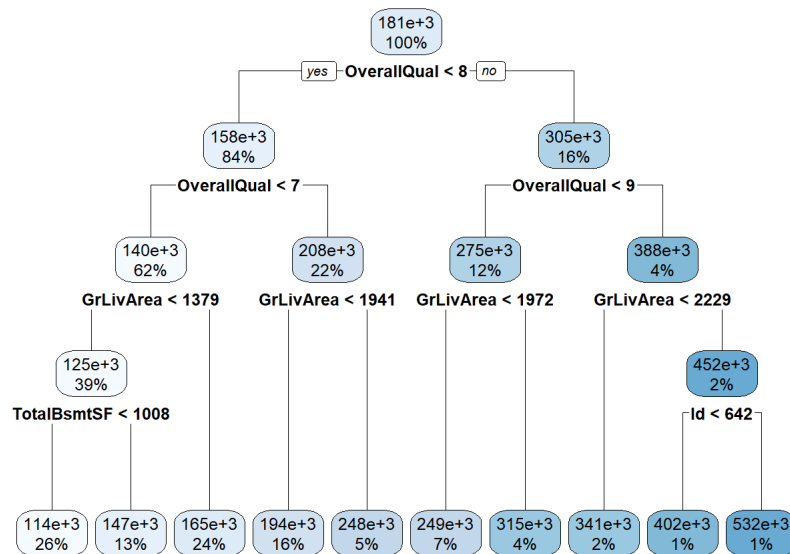
## CART
##
## 1460 samples
## 80 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1313, 1315, 1315, 1314, 1312, 1314, ...
## Resampling results across tuning parameters:
##
## maxdepth RMSE Rsquared MAE
## 2 51790.47 0.5799916 36288.11
## 3 48714.86 0.6305755 34714.28
## 4 47727.38 0.6503320 32980.37
## 5 47796.33 0.6519421 32614.72
## 6 46842.78 0.6652085 31924.90
## 7 45771.90 0.6829139 31439.35
## 8 45036.55 0.6944306 30361.97
## 9 44978.15 0.6972515 30434.04
## 10 45032.94 0.6963769 30346.85
## 11 45032.94 0.6963769 30346.85
## 12 45032.94 0.6963769 30346.85
## 13 45032.94 0.6963769 30346.85
## 14 45032.94 0.6963769 30346.85
## 15 45032.94 0.6963769 30346.85
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was maxdepth = 9.

```

```
plot(modelo7.1)
```




```
rpart.plot(modelo7.1$finalModel)
```



```

predModelo7.1 <- predict(modelo7.1, newdata = datos)
rmseModelo7.1test<-RMSE(predModelo7.1,mergeprice)
mseModelo7.1test<-mean((predModelo7.1 - mergeprice)^2)

predModelo7.1Train <- predict(modelo7.1, newdata = train)
rmseModelo7.1train<-RMSE(predModelo7.1Train,train$SalePrice)
mseModelo7.1train<-mean((predModelo7.1Train - train$SalePrice)^2)

```

```
mseModelo7.1test
```

```
## [1] 5843740252
```

```
mseModelo7.1train
```

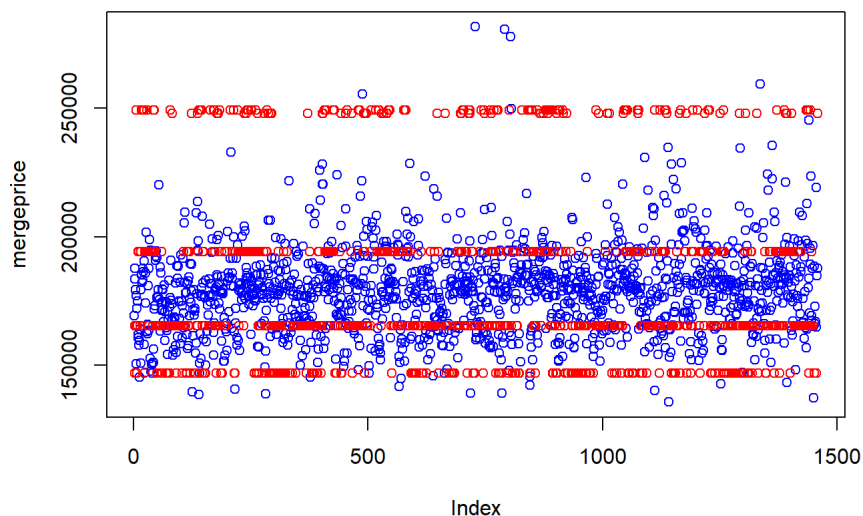
```
## [1] 1542464278
```

```

plot(mergeprice,col="blue", main="Predicciones vs valores originales")
points(predModelo7.1, col="red")
legend(30,45,legend=c("original", "prediccion"),col=c("blue", "red"),pch=1, cex=0.8)

```

Predicciones vs valores originales



```

profundidades <- expand.grid(maxdepth = 2:20)

system.time (modelo8<-train(SalePrice~.,
                           data=train,
                           method="rpart2",
                           tuneLength = 10,
                           tuneGrid = profundidades,
                           trControl = controlcv,
                           metric = "RMSE",
                           na.action = na.pass
                           )
)

```

```

##    user  system elapsed
##   5.00    0.17    5.30

```

```

modelo8

```

```

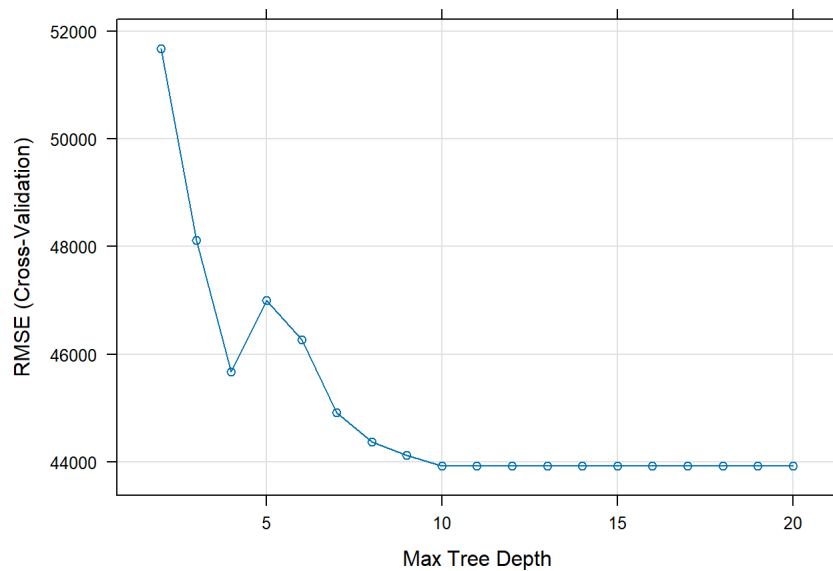
## CART
##
## 1460 samples
##   80 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1312, 1315, 1313, 1315, 1314, 1315, ...
## Resampling results across tuning parameters:
##
##  maxdepth  RMSE      Rsquared  MAE
##    2      51680.00  0.5800860  36273.05
##    3      48114.30  0.6374921  34517.84
##    4      45672.44  0.6749482  32320.24
##    5      46999.00  0.6595927  32437.95
##    6      46267.24  0.6703882  32203.79
##    7      44912.76  0.6920051  31285.49
##    8      44371.10  0.7013002  30326.36
##    9      44120.03  0.7042711  29808.43
##   10      43921.16  0.7085804  29611.42
##   11      43921.16  0.7085804  29611.42
##   12      43921.16  0.7085804  29611.42
##   13      43921.16  0.7085804  29611.42
##   14      43921.16  0.7085804  29611.42
##   15      43921.16  0.7085804  29611.42
##   16      43921.16  0.7085804  29611.42
##   17      43921.16  0.7085804  29611.42
##   18      43921.16  0.7085804  29611.42
##   19      43921.16  0.7085804  29611.42
##   20      43921.16  0.7085804  29611.42
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was maxdepth = 10.

```

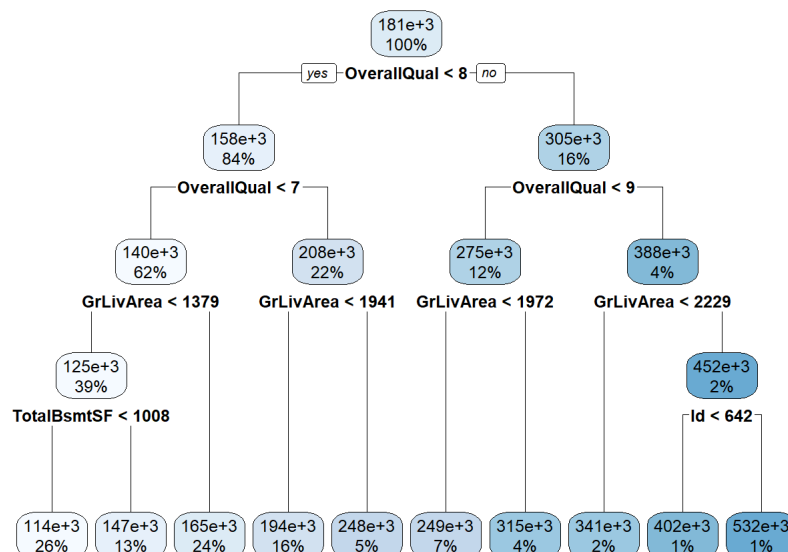
```

plot(modelo8)

```



```
rpart.plot(modelo8$finalModel)
```



```

predModelo8 <- predict(modelo8, newdata = datos)
rmseModelo8test<-RMSE(predModelo8,mergeprice)
mseModelo8test<-mean((predModelo8 - mergeprice)^2)

predModelo8Train <- predict(modelo8, newdata = train)
rmseModelo8train<-RMSE(predModelo8Train,train$SalePrice)
mseModelo8train<-mean((predModelo8Train - train$SalePrice)^2)

```

```
mseModelo8test
```

```
## [1] 5843740252
```

```
mseModelo8train
```

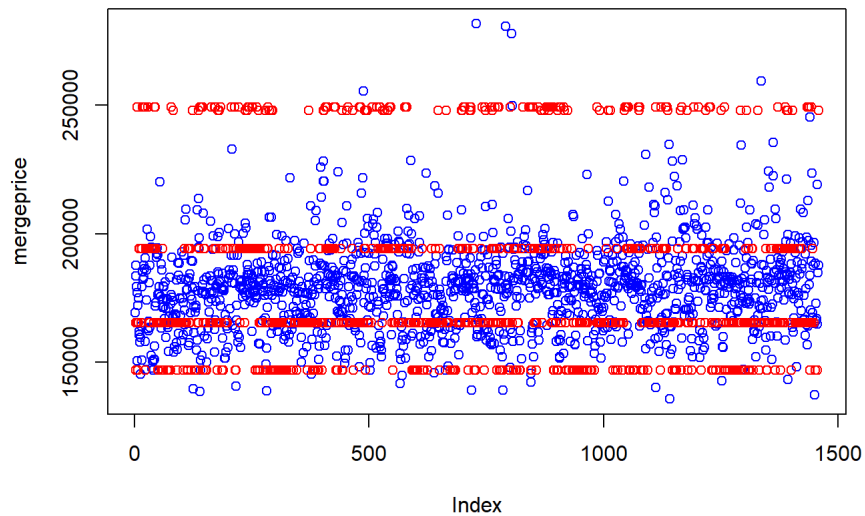
```
## [1] 1542464278
```

```

plot(mergeprice,col="blue", main="Predicciones vs valores originales")
points(predModelo8, col="red")
legend(30,45,legend=c("original", "prediccion"),col=c("blue", "red"),pch=1, cex=0.8)

```

Predicciones vs valores originales



```
profundidades <- expand.grid(maxdepth = 2:3)

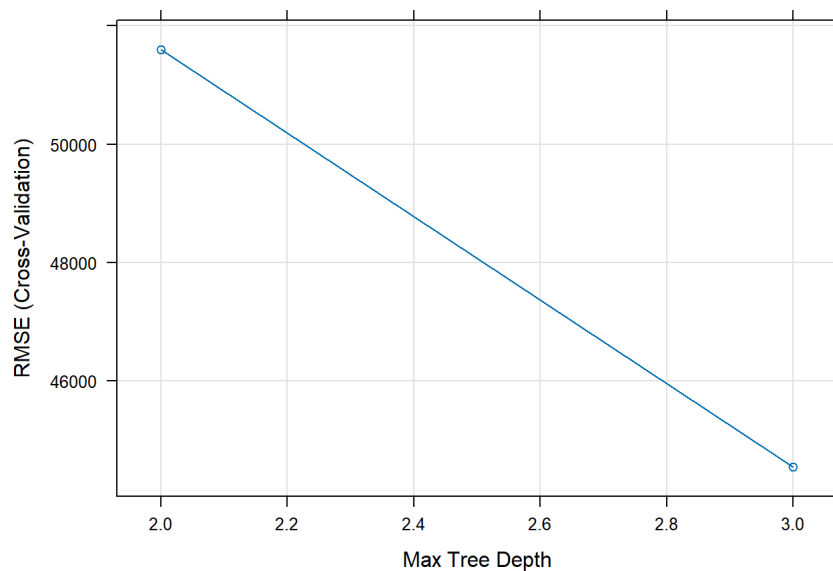
system.time (modelo9<-train(SalePrice~.,
  data=train,
  method="rpart2",
  tuneLength = 10,
  tuneGrid = profundidades,
  trControl = controlcv,
  metric = "RMSE",
  na.action = na.pass
))
```

```
## user system elapsed
## 1.23 0.01 1.31
```

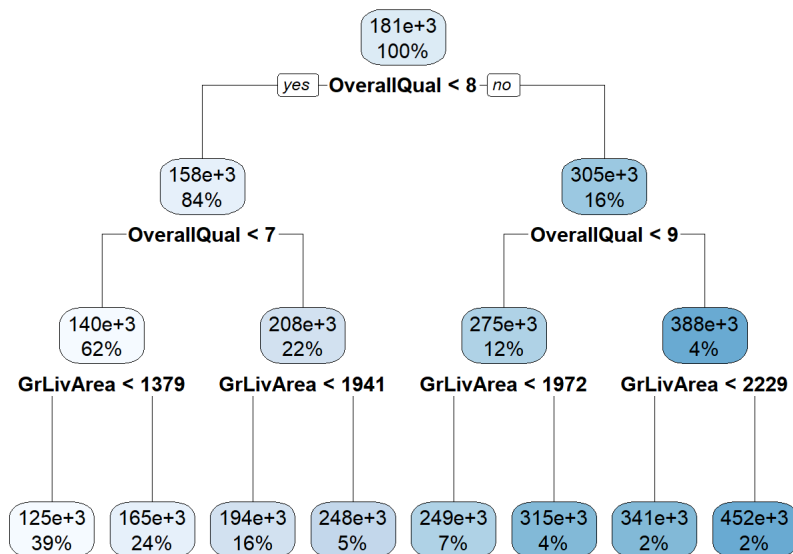
```
modelo9
```

```
## CART
##
## 1460 samples
## 80 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 1313, 1315, 1315, 1314, 1315, 1314, ...
## Resampling results across tuning parameters:
##
## maxdepth RMSE Rsquared MAE
## 2 51600.32 0.5795800 36247.96
## 3 44546.45 0.6944071 30922.63
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was maxdepth = 3.
```

```
plot(modelo9)
```



```
rpart.plot(modelo9$finalModel)
```



```

predModelo9 <- predict(modelo9, newdata = datos)
rmseModelo9test<-RMSE(predModelo9,mergeprice)
mseModelo9test<-mean((predModelo9 - mergeprice)^2)

predModelo9Train <- predict(modelo9, newdata = train)
rmseModelo9train<-RMSE(predModelo9Train,train$SalePrice)
mseModelo9train<-mean((predModelo9Train - train$SalePrice)^2)

```

```
mseModelo9test
```

```
## [1] 4741037550
```

```
mseModelo9train
```

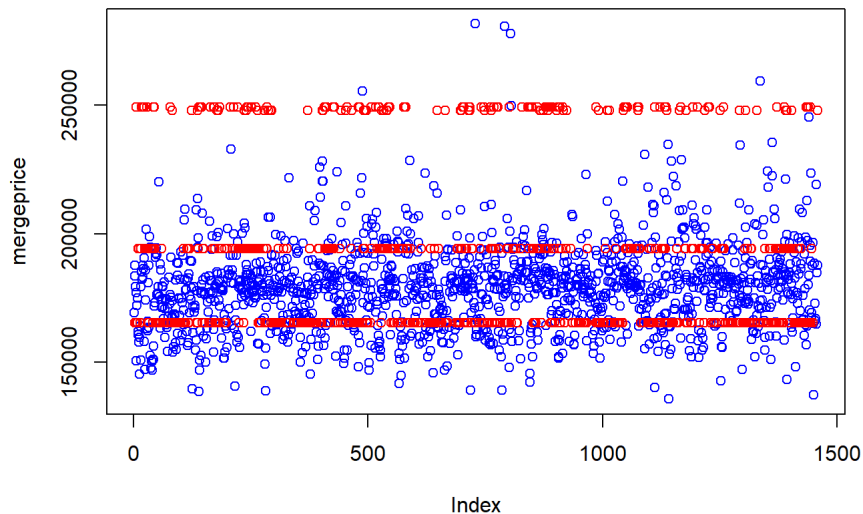
```
## [1] 1706990759
```

```

plot(mergeprice,col="blue", main="Predicciones vs valores originales")
points(predModelo9, col="red")
legend(30,45,legend=c("original", "prediccion"),col=c("blue", "red"),pch=1, cex=0.8)

```

Predicciones vs valores originales



```
modeloRF1<-randomForest(SalePrice~.,data=train)
prediccionRF1<-predict(modeloRF1,newdata = datos)
rmseModelo10test<-RMSE(prediccionRF1,mergeprice)
mseModelo10test<-mean((prediccionRF1 - mergeprice)^2)

prediccionRF1Train<-predict(modeloRF1,newdata = datos)
rmseModelo10train<-RMSE(prediccionRF1Train,train$SalePrice)
```

```
## Warning in pred - obs: longer object length is not a multiple of shorter object
## length
```

```
mseModelo10train<-mean((prediccionRF1Train - train$SalePrice)^2)
```

```
## Warning in prediccionRF1Train - train$SalePrice: longer object length is not a
## multiple of shorter object length
```

```
mseModelo10test
```

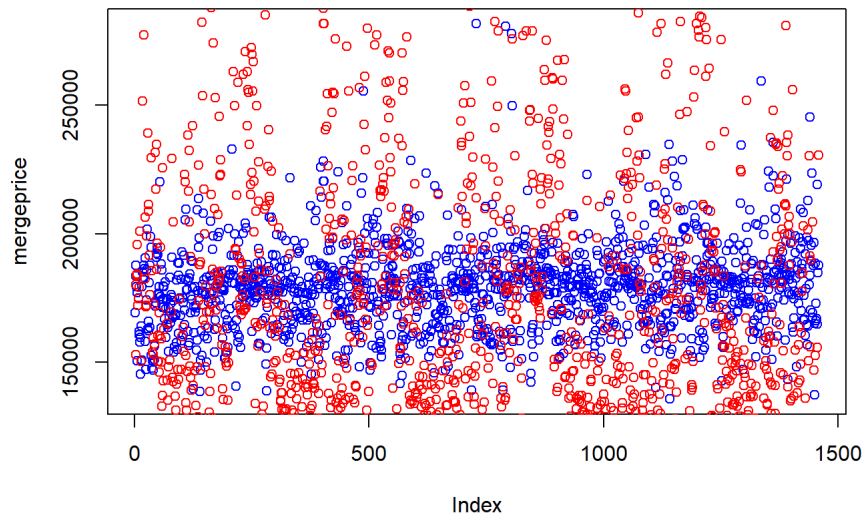
```
## [1] 4731196302
```

```
mseModelo10train
```

```
## [1] 11338402180
```

```
plot(mergeprice,col="blue", main="Predicciones vs valores originales")
points(prediccionRF1, col="red")
legend(30,45,legend=c("original", "prediccion"),col=c("blue", "red"),pch=1, cex=0.8)
```

Predicciones vs valores originales



```
CompModelos<-c(rmseModelo6.1test,rmseModelo7.1test,rmseModelo8test,rmseModelo9test,rmseModelo10test)
names(CompModelos)<-c("Tuneado Modelo1","Tuneado Modelo2","Tuneado Modelo3", "Tuneado Modelo4","Modelo Random Forest")
CompModelos
```

##	Tuneado Modelo1	Tuneado Modelo2	Tuneado Modelo3
##	76444.36	76444.36	76444.36
##	Tuneado Modelo4	Modelo Random Forest	
##	68855.19	68783.69	

Con la comparación de los 4 modelos utilizados con tuneo y el modelo de Random Forest podemos observar gráficamente con el modelo de Random Forest que tiene pocas asimilaciones con el modelo pero observamos que los modelo tuneado tienen mayor rmse a comparación del modelo de Random Forest. POr lo tanto inferimos aunque tenga menor rmse que el modelo 1 y 2, observamos que es mejor tener el modelo de Random Forest ya que no tiene overfitting de datos y esta cerca del rango de los modelos de tuneado.